

Relazione Progetto: File System Statistics (FSStat)

Programmazione Concorrente e Distribuita

Alessandro Martini, Alessandro Torelli

Anno Accademico 2025/2026

Sommario

Il presente documento descrive la progettazione e l'implementazione della libreria **FSStatLib**, sviluppata in Java 21+. L'obiettivo del progetto è analizzare un albero di directory del file system, calcolando il numero totale di file e la loro distribuzione in fasce di dimensione. L'elaborazione è stata parallelizzata utilizzando tre metodi e tecnologie differenti: i Virtual Threads, programmazione asincrona basata su event loop, e la programmazione reattiva con RX.

Indice

1	Introduzione e Requisiti	2
2	Analisi del Problema e Scelte Concorrenti	2
2.1	Approccio con i Virtual Threads	3
2.2	Approccio Asincrono basato su Event Loop	4
2.3	Approccio Reattivo con RX	5
3	Architettura e Design	6
3.1	Design dell'implementazione Virtual Threads	6
3.2	Design dell'implementazione Event Loop	8
3.3	Design dell'implementazione RX	12

1 Introduzione e Requisiti

Obiettivo

L'obiettivo è la realizzazione della libreria `FSStatLib`, che espone un metodo asincrono `getFSReport` per l'analisi statistica delle dimensioni dei file contenuti in un albero di directory del file system. Dato un percorso radice `D`, il metodo calcola e restituisce in modo asincrono un report `R` contenente:

- il **numero totale di file** presenti in `D`, incluse ricorsivamente tutte le sottodirectory;
- la **distribuzione delle dimensioni** dei file: dato un valore massimo di dimensione `MaxFS` e un numero di bande `NB`, il range `[0, MaxFS]` viene suddiviso in `NB` intervalli di ampiezza uniforme; per ciascun intervallo viene contato il numero di file la cui dimensione vi rientra. Viene inoltre conteggiata separatamente la quantità di file con dimensione superiore a `MaxFS`. Il report include quindi complessivamente `NB + 1` contatori.

I parametri `D`, `MaxFS` e `NB` sono forniti dal chiamante come argomenti del metodo.

Versioni implementate

Il progetto è stato declinato in **tre versioni indipendenti**, ciascuna aderente a una specifica disciplina di programmazione concorrente discussa durante il corso:

1. **Virtual Threads** (Java): paradigma *thread-per-task* con Virtual Thread leggeri gestiti dalla JVM.
2. **Programmazione asincrona basata su Event Loop**: composizione di operazioni non bloccanti.
3. **Programmazione reattiva con RX**: modellazione del file system traversal come stream di eventi, elaborato tramite operatori funzionali (RxJava).

Un vincolo di progetto esplicito impone che le tre versioni siano sviluppate in modo **indipendente**, senza generalizzare o riutilizzare codice tra di esse qualora ciò compromettesse la purezza della disciplina di programmazione propria di ciascun approccio.

Requisito opzionale

Il progetto prevede un requisito opzionale `[*]`, obbligatorio per il conseguimento della lode, che estende la libreria con funzionalità interattive:

- possibilità di **interrompere** la generazione del report in qualsiasi momento;
- **aggiornamenti dinamici** delle statistiche durante la computazione, visualizzabili in tempo reale.

A supporto di queste funzionalità è prevista una **GUI minimale**, dotata di controlli per avviare e interrompere la scansione e di un'area di visualizzazione per il monitoraggio progressivo del report.

2 Analisi del Problema e Scelte Concorrenti

L'attraversamento ricorsivo di un file system e l'interrogazione delle proprietà dei file sono tipiche operazioni legate all'I/O. In questi scenari di I/O-bound, l'uso intensivo dei classici thread di piattaforma presenta gravi limiti di scalabilità. Per esplorare soluzioni diverse a questo problema di concorrenza, il progetto è stato declinato in tre paradigmi distinti.

2.1 Approccio con i Virtual Threads

I Virtual Threads, introdotti stabilmente in Java 21, sono thread *user-mode* gestiti dalla JVM anziché dal sistema operativo. Rispetto ai *platform thread*, il loro overhead di creazione e il consumo di memoria sono trascurabili, il che li rende adatti a un modello *Thread-per-task*: è possibile istanziarne decine di migliaia senza incorrere in `OutOfMemoryError` né in degrado delle prestazioni.

Questa caratteristica si adatta in modo naturale all'attraversamento ricorsivo di un albero di directory: la struttura del file system è intrinsecamente ramificata e imprevedibile, e ogni nodo (directory) richiede operazioni di I/O bloccanti a latenza variabile (apertura dello stream, lettura degli attributi dei file con `Files.size()`). Assegnare un Virtual Thread a ciascun nodo permette di sfruttare appieno la concorrenza senza dover dimensionare a priori un pool.

Struttura dell'implementazione. Il punto di ingresso pubblico è il metodo `getFSReport()`, che restituisce una `CompletableFuture<FSReport>` per offrire un'interfaccia non bloccante al chiamante (CLI o GUI). Internamente, la computazione reale viene avviata tramite `CompletableFuture.supplyAsync()` che esegue il corpo su un Virtual Thread. All'interno di questo contesto viene creato un `ExecutorService` dedicato tramite `Executors.newVirtualThreadPerTaskExecutor()`, racchiuso in un blocco `try-with-resources`:

```
1 try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {  
2     Path rootPath = Paths.get(directoryPath);  
3     return processDirectory(rootPath, maxFS, nb, executor).get();  
4 }
```

La chiusura automatica dell'executor al termine del blocco `try` ha una semantica cruciale: `ExecutorService.close()`, (che viene invocato automaticamente nel blocco `try-with-resources`) invoca `shutdown()` seguito da `awaitTermination()`, garantendo che nessun task figlio venga abbandonato prima che il risultato finale sia disponibile. Questo meccanismo funge da *barriera di sincronizzazione* implicita su tutto l'albero dei task.

Ricorsione concorrente e Map-Reduce. Il metodo `processDirectory()` incarna il pattern *divide et impera*: per ogni nodo dell'albero viene sottomesso all'executor un nuovo task (e quindi un nuovo Virtual Thread). All'interno del task, si itera sul contenuto della directory tramite `Files.newDirectoryStream()`:

- Per ogni **file regolare**: si invoca `Files.size()` (operazione I/O bloccante) e si aggiorna un oggetto `FSReport locale` al Virtual Thread corrente.
- Per ogni **sottodirectory**: si sottomette ricorsivamente un nuovo task all'executor, raccogliendo il `Future<FSReport>` restituito in una lista di *subTask*.

Terminata la scansione della directory corrente, il Virtual Thread chiama `task.get()` su ciascun *subTask* per attenderne il completamento e fondere i risultati parziali nel report locale tramite il metodo `FSReport.merge()`:

```
1 for (Future<FSReport> task : subTasks) {  
2     localReport.merge(task.get());  
3 }
```

Questa fase di riduzione segue un pattern *Map-Reduce*: la fase di *map* distribuisce il lavoro di scansione su Virtual Thread indipendenti, mentre la fase di *reduce* aggrega i report parziali bottom-up lungo l'albero, propagando i risultati verso il nodo radice.

Assenza di contesa e gestione dell'I/O bloccante. Una proprietà rilevante di questo design è la completa assenza di stato condiviso mutabile tra Virtual Thread concorrenti: ogni task opera esclusivamente sul proprio `FSReport` locale. La fusione dei risultati (`merge`) avviene sempre in modo sequenziale nel Virtual Thread padre, dopo che i figli sono terminati. Ne consegue che non è necessario alcun meccanismo di sincronizzazione esplicito (né `synchronized` né `ReentrantLock`) per proteggere il report.

Quando un Virtual Thread si blocca su un'operazione di I/O — ad esempio `Files.size()` o `task.get()` in attesa di un risultato non ancora disponibile — la JVM effettua automaticamente l'*unmount* del Virtual Thread dal *carrier thread* (il platform thread sottostante), che viene immediatamente liberato per eseguire un altro Virtual Thread pronto. Questo meccanismo permette al ridotto pool di carrier thread (tipicamente uguale al numero di core fisici) di servire migliaia di Virtual Thread concorrenti, eliminando il costo del context switch a livello di sistema operativo.

2.2 Approccio Asincrono basato su Event Loop

Il secondo paradigma adottato si basa sul modello *Event Loop*, incarnato in questa implementazione dal runtime **Node.js** (scritto in TypeScript). In questo modello, un **singolo thread JavaScript** esegue il codice applicativo, delegando tutte le operazioni di I/O al layer sottostante *libuv*, che le gestisce tramite un proprio thread pool interno al runtime. Al completamento di ciascuna operazione, la relativa callback viene accodata nell'event loop ed eseguita dal thread principale non appena questo è libero.

Adattamento al problema. L'attraversamento ricorsivo di un albero di directory è un workload *I/O-bound* con un elevato grado di parallelismo naturale: i contenuti di directory diverse sono indipendenti tra loro e possono essere letti concorrentemente. Questo si adatta perfettamente al modello event loop, dove la concorrenza non è ottenuta tramite thread multipli ma tramite l'*interleaving* di operazioni asincrone: il thread principale non attende mai il completamento di una lettura su disco, ma registra una `Promise` e prosegue a processare altri eventi già completati.

Concorrenza tramite `Promise.all`. Il cuore della concorrenza risiede nel metodo `traverseDirectory` di `FsStatScannerImpl`. Dopo aver letto il contenuto di una directory tramite l'API non bloccante `fs.readdir()` (da `fs/promises`), il metodo separa i risultati in file e sottodirectory, quindi lancia *simultaneamente* tutte le operazioni figlie tramite `Promise.all`:

```
1 await Promise.all([
2   ...files.map((file) => exploreFile(file.name)),
3   ...directories.map((dir) => exploreDir(dir.name)),
4 ]);
```

Questo produce un *fan-out* parallelo: tutte le chiamate a `fs.stat()` sui file e tutte le esplorazioni ricorsive delle sottodirectory vengono avviate in parallelo, senza attendere che una termini prima di avviare la successiva. La ricorsione si propaga in profondità nell'albero mantenendo questo stesso pattern ad ogni livello.

Assenza di sincronizzazione per costruzione. Una proprietà fondamentale di questo approccio è che l'aggiornamento dello stato condiviso — il metodo `updateReport(fileSize)` su `FsStatReportImpl` — avviene **senza alcun meccanismo di sincronizzazione** (niente lock, niente mutex, niente operazioni atomiche). Questo è sicuro per costruzione grazie alla *run-to-completion semantics* di JavaScript: l'event loop garantisce che ogni callback venga eseguita interamente prima che un'altra possa iniziare. Due invocazioni di `updateReport` non possono mai sovrapporsi temporalmente, rendendo strutturalmente impossibile qualsiasi race condition sullo stato del report.

Gestione degli errori e robustezza. L'implementazione distingue due categorie di errori di I/O. Gli errori di accesso (EACCES, EPERM) vengono silenziosamente ignorati tramite `.catch()`, permettendo alla scansione di proseguire sui nodi raggiungibili. Gli errori strutturali (ENOENT, ENOTDIR) vengono invece convertiti in eccezioni tipizzate e propagati al chiamante, causando il rigetto della `Promise` restituita da `getReport`. È inoltre presente una *blacklist* di path di sistema (`/proc`, `/sys`, `/dev` e altri) che vengono esclusi preventivamente dalla scansione per evitare loop infiniti o comportamenti non deterministici su filesystem virtuali.

2.3 Approccio Reattivo con RX

La terza tecnologia utilizza la Programmazione Reattiva, nella sua declinazione **RxJava**, un paradigma orientato ai flussi di dati (*data stream*) e alla loro trasformazione dichiarativa tramite operatori funzionali. A differenza dei Virtual Thread e dell'Event Loop, dove la concorrenza è governata esplicitamente dal programmatore (sottomissione di task, composizione di `Promise`), nel modello reattivo è la libreria stessa a occuparsi dello scheduling: il compito dello sviluppatore si riduce a descrivere *cosa* deve accadere ai dati che attraversano la pipeline, non *come* e *su quale thread* ciò debba avvenire.

Adattamento al problema. L'attraversamento del file system viene modellato come un flusso di eventi: ogni file incontrato durante la visita ricorsiva dell'albero di directory viene emesso come elemento di un `Flowable<File>`. Si tratta di un workload che presenta due nature distinte, ciascuna gestita da un meccanismo RX differente:

- la **produzione** degli eventi (l'attraversamento delle directory tramite `File.listFiles()`) è un'operazione I/O-bound, intrinsecamente sequenziale nella sua fase di scoperta dei nodi dell'albero;
- il **consumo** degli eventi (la classificazione di ciascun file nella fascia di dimensione corretta e l'aggiornamento del report) è invece un'operazione CPU-bound, priva di dipendenze tra un file e l'altro, e quindi parallelizzabile senza vincoli.

Questa distinzione ha guidato la scelta degli operatori: la generazione dello stream avviene tramite un *Publisher* custom, mentre l'elaborazione a valle sfrutta l'operatore `parallel()` di RxJava per distribuire il carico di classificazione su più thread.

Backpressure e disaccoppiamento produttore-consumatore. Poiché la fase di scoperta dei file può produrre eventi più velocemente di quanto la pipeline a valle riesca a consumarli (specialmente su alberi di directory molto popolati), è necessario un meccanismo di *backpressure* che regoli il flusso tra produttore e consumatore evitando un consumo di memoria incontrollato o eccezioni di overflow. RxJava affronta questo problema tramite il tipo `Flowable`, che a differenza di `Observable` implementa nativamente il protocollo *Reactive Streams*.

Cancellazione cooperativa. Un ulteriore vantaggio del modello reattivo, rilevante per il requisito opzionale di interruzione della scansione, è che ogni stream RX è *cancellabile*: disiscrivere da un `Flowable` (o disporre di una `Disposable`) propaga automaticamente un segnale di cancellazione fino alla sorgente. Il produttore può interrogare periodicamente questo stato per interrompere tempestivamente un lavoro non più necessario, senza dover implementare a mano alcun meccanismo di interrupt flag condiviso, come sarebbe invece necessario con i thread tradizionali.

3 Architettura e Design

3.1 Design dell'implementazione Virtual Threads

L'implementazione con Virtual Threads è articolata in due classi principali nel package `it.unibo.fsstat.lib`: `FSReport`, che modella il risultato della computazione, e `FSStatLib`, che ne orchestra l'esecuzione concorrente. La CLI (`Main`) interagisce esclusivamente con l'interfaccia pubblica di `FSStatLib`, rimanendo ignara del paradigma di concorrenza sottostante.

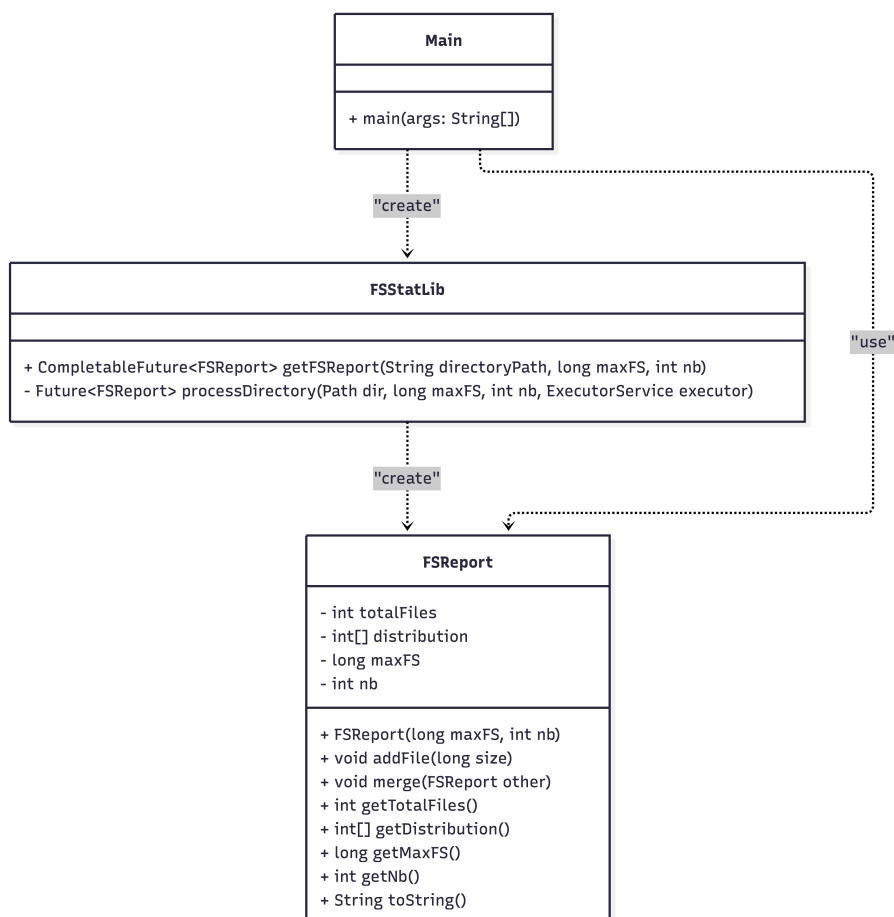


Figura 1: Diagramma UML delle classi dell'implementazione con Virtual Threads.

Il modello dati: `FSReport`. `FSReport` è un *value object* che incapsula tre informazioni: il conteggio totale dei file (`totalFiles`), i parametri di classificazione (`maxFS` e `nb`), e un array `distribution` di dimensione `nb + 1`. Le prime `nb` celle corrispondono alle bande di dimensione nell'intervallo `[0, maxFS]`, mentre la cella di indice `nb` raccoglie i file che superano `maxFS`.

La classificazione di un singolo file avviene nel metodo `addFile(long size)`, che calcola l'indice di banda tramite divisione intera:

```
1 double step = (double) maxFS / nb;
2 int bandIndex = (int) (size / step);
3 if (bandIndex == nb) bandIndex = nb - 1; // Caso limite: size == maxFS
```

Il metodo `merge(FSReport other)` somma componente per componente i due array `distribution` e accumula i conteggi di `totalFiles`. Questa operazione è la chiave del pattern Map-Reduce: è associativa e non richiede alcuna sincronizzazione perché viene sempre invocata in modo sequenziale dal Virtual Thread padre dopo che i figli sono terminati.

La libreria: FSStatLib. L'interfaccia pubblica espone un unico metodo:

```
1 public CompletableFuture<FSReport> getFSReport(  
2     String directoryPath, long maxFS, int nb)
```

La restituzione di una `CompletableFuture` disaccoppia il chiamante dalla concorrenza interna: la CLI può concatenare callback (`thenAccept`) oppure attendere sincronamente con `join()`, senza dover conoscere né i Virtual Thread né l'executor sottostante.

Internamente, la computazione viene incapsulata in una `CompletableFuture.supplyAsync()`. All'interno di questo contesto asincrono viene creato un `ExecutorService` dedicato ai Virtual Threads tramite `Executors.newVirtualThreadPerTaskExecutor()`:

```
1 try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {  
2     Path rootPath = Paths.get(directoryPath);  
3     return processDirectory(rootPath, maxFS, nb, executor).get();  
4 }
```

Il blocco `try-with-resources` ha una funzione di lifecycle management precisa: alla sua uscita, `ExecutorService.close()` invoca `shutdown()` e `awaitTermination()`, assicurando che tutti i Virtual Thread lanciati ricorsivamente abbiano completato la propria esecuzione prima che il risultato venga restituito alla `CompletableFuture`.

Il flusso ricorsivo: processDirectory. `processDirectory` è il cuore dell'algoritmo. Ogni sua invocazione sottomette all'executor un nuovo task — e quindi un nuovo Virtual Thread — incaricato di elaborare una singola directory. Il flusso interno al task segue tre fasi:

1. **Scansione locale:** si itera sul contenuto della directory tramite `Files.newDirectoryStream()`. Per ogni file regolare, `Files.size()` legge la dimensione dal disco (operazione I/O bloccante) e la classifica nel `FSReport` locale al thread. Per ogni sottodirectory, si invoca ricorsivamente `processDirectory`, raccogliendo il `Future<FSReport>` restituito in una lista (`subTasks`).
2. **Riduzione (join):** terminata la scansione, il Virtual Thread chiama `task.get()` su ciascun elemento di `subTasks`. Se il task figlio non è ancora completato, il Virtual Thread si sospende — effettuando l'*unmount* dal carrier thread — e si risveglia non appena il risultato è disponibile. I report parziali vengono fusi nel report locale tramite `merge()`.
3. **Restituzione:** il report locale, ora completo di tutti i contributi dei sottoalberi, viene restituito come valore del `Callable` sottomesso all'executor.

La struttura risultante è un albero di Virtual Thread isomorfo all'albero delle directory: ogni nodo dell'albero FS corrisponde a un task concorrente, e la riduzione avviene bottom-up per propagazione dei `Future`.

Gestione degli errori. Gli errori di accesso al file system (directory protette, symlink non risolvibili) vengono catturati localmente da un blocco `catch (IOException e)` all'interno di ciascun task: la directory inaccessibile viene saltata con un messaggio su `stderr`, mentre la computazione prosegue sui nodi raggiungibili. Eventuali eccezioni non gestite che sfuggono al task vengono incapsulate in una `CompletionException` e propagate al chiamante tramite la `CompletableFuture`, che entra nello stato *exceptionally completed* e può essere intercettata con `.exceptionally()`.

Integrazione con la CLI. In `Main`, il chiamante configura i tre parametri della scansione (`directoryPath`, `maxFS`, `nb`) e concatena alla `CompletableFuture` una callback `thenAccept` per la stampa del report. La chiamata `.join()` finale blocca il thread principale fino al completamento, evitando che il programma termini prima che la computazione asincrona sia conclusa.

La formattazione del report calcola le etichette delle bande moltiplicando l'indice per `bandSize` = `maxFS / nb`, producendo intervalli del tipo `[i·bandSize, (i+1)·bandSize]`.

3.2 Design dell'implementazione Event Loop

L'implementazione con Event Loop è sviluppata in **TypeScript** sul runtime **Node.js**, ed è organizzata in moduli con responsabilità ben separate. A differenza della versione Java con Virtual Threads, l'intera concorrenza è ottenuta senza istanziare alcun thread esplicitamente: il parallelismo emerge dalla composizione di **Promise** gestite dall'event loop di Node.js e dal thread pool di *libuv*.

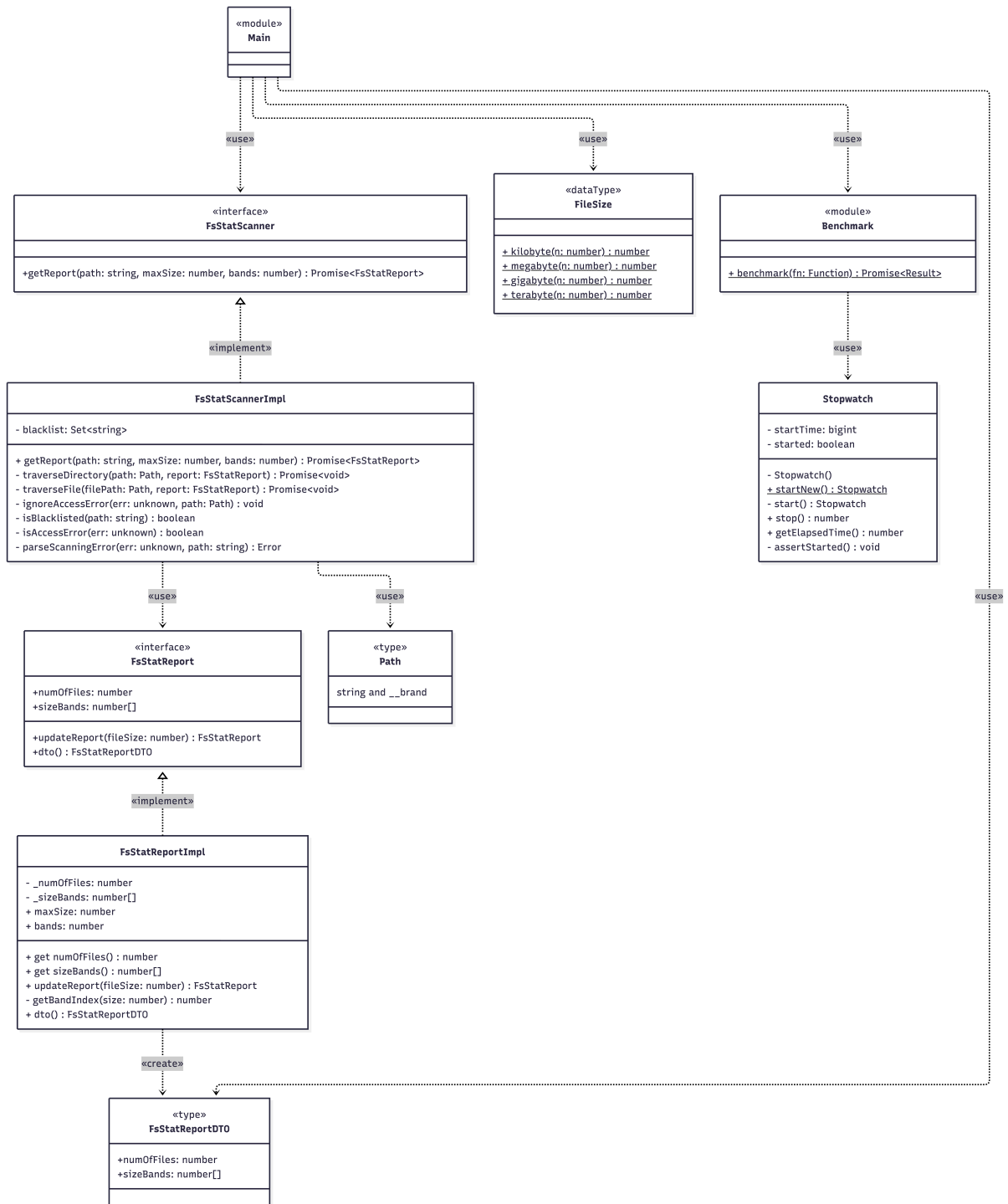


Figura 2: Diagramma UML delle classi dell'implementazione con Event Loop (TypeScript).

Struttura dei moduli. Il codebase è suddiviso in moduli distinti, ciascuno con una responsabilità unica:

- **report.ts** — modella il report statistico e la sua logica di aggiornamento;
- **scanner.ts** — implementa la logica di traversal asincrono del file system;
- **path.ts** — fornisce un tipo validato per i percorsi del file system;

- `sizes.ts` — utility per la conversione delle unità di dimensione dei file;
- `benchmark.ts` — utility per la misurazione dei tempi di esecuzione;
- `index.ts` — façade pubblica della libreria verso i consumer esterni.

Pattern di incapsulamento: factory function e interfaccia pubblica. Sia `FsStatReport` che `FsStatScanner` adottano un pattern idiomatico TypeScript per l'information hiding: la classe concreta (`FsStatReportImpl`, `FsStatScannerImpl`) è dichiarata `class` privata al modulo e non viene mai esportata. L'unico punto di accesso è la *factory function* omonima, che restituisce l'interfaccia pubblica:

```
1 export function FsStatScanner(): FsStatScanner {
2   return new FsStatScannerImpl();
3 }
```

Il consumer non conosce mai la classe concreta né può accedere ai suoi membri privati: interagisce esclusivamente tramite l'interfaccia, che espone solo i metodi necessari (`getReport`, `updateReport`, `dto`). Questo garantisce un disaccoppiamento totale tra interfaccia e implementazione.

Il Branded Type Path. Il modulo `path.ts` definisce un *Branded Type* per i percorsi del file system:

```
1 export type Path = string & { readonly __brand: unique symbol };
```

Si tratta di un *phantom type*: a runtime `Path` è semplicemente una stringa, ma a compile-time il type system di TypeScript impone che solo path prodotti da `createPath()` o `concatPaths()` — che normalizzano e validano l'input — possano essere passati alle funzioni interne dello scanner. Qualsiasi stringa grezza produce un errore di tipo a compile-time, eliminando un'intera categoria di bug senza alcun overhead a runtime.

Separazione del modello interno dall'API esterna: il metodo `dto()`. `FsStatReportImpl` mantiene internamente uno stato mutabile (`_numOfFiles`, `_sizeBands`), aggiornato ad ogni chiamata di `updateReport()`. Tuttavia, il consumer esterno non riceve mai accesso diretto a questo stato: il metodo `dto()` produce uno snapshot immutabile nel formato `FsStatReportDTO`, definito come tipo separato nel package dei tipi condivisi:

```
1 dto(): FsStatReportDTO {
2   return {
3     numOfFiles: this.numOfFiles,
4     sizeBands: [...this.sizeBands], // copia difensiva
5   };
6 }
```

La copia difensiva dell'array (`[...this.sizeBands]`) garantisce che il DTO sia uno snapshot indipendente: eventuali aggiornamenti successivi al report non alterano il risultato già consegnato al consumer.

Il flusso di traversal: `FsStatScannerImpl`. Il metodo pubblico `getReport()` istanzia un `FsStatReport` vuoto e avvia la traversal ricorsiva tramite `traverseDirectory()`, restituendo la `Promise` che si risolve al termine dell'intera scansione.

Il metodo `traverseDirectory()` costituisce il nucleo del design asincrono. Il suo flusso segue tre fasi:

1. **Guard sulla blacklist:** prima di qualsiasi operazione I/O, il path viene verificato contro un `Set` di percorsi di sistema esclusi (`/proc`, `/sys`, `/dev`, `/run` e altri). Se il path è nella

blacklist, la funzione ritorna immediatamente senza effettuare alcuna lettura, prevenendo loop infiniti o comportamenti non deterministici su filesystem virtuali.

2. **Lettura della directory:** `fs.readdir()` con l'opzione `withFileTypes: true` restituisce direttamente le informazioni sul tipo di ciascuna voce (file o directory), evitando una chiamata aggiuntiva a `fs.stat()` per discriminarle. In caso di errore di accesso (EACCES/EPERM), il `.catch()` restituisce un array vuoto, permettendo alla scansione di proseguire sui nodi accessibili.
3. **Fan-out parallelo con `Promise.all`:** file e sottodirectory vengono processati simultaneamente in un'unica barriera di sincronizzazione:

```
1 await Promise.all([
2   ...files.map((file) => exploreFile(file.name)),
3   ...directories.map((dir) => exploreDir(dir.name)),
4 ]);
```

Tutte le chiamate a `fs.stat()` sui file e tutte le esplorazioni ricorsive delle sottodirectory vengono avviate in parallelo. L'event loop gestisce il completamento di ciascuna operazione in modo indipendente: non appena `fs.stat()` completa per un file, la relativa callback aggiorna il report senza attendere gli altri. `Promise.all` funge da barriera: `traverseDirectory` si risolve solo quando *tutte* le operazioni figlie sono terminate.

Gestione degli errori a due livelli. L'implementazione distingue esplicitamente due categorie di errori tramite il metodo `ignoreAccessError()`:

- **Errori di permesso** (EACCES, EPERM): vengono silenziosamente ignorati. La scansione prosegue sui path accessibili, poiché è normale che alcune directory di sistema siano protette.
- **Errori strutturali** (ENOENT, ENOTDIR): vengono convertiti in eccezioni tipizzate tramite `parseScanningError()` e propagati al consumer come rigetto della `Promise` restituita da `getReport()`, segnalando una condizione anomala che richiede attenzione.

La façade pubblica: `index.ts`. Il modulo `index.ts` funge da unico punto di accesso alla libreria per i consumer esterni. Crea internamente un'unica istanza di `FsStatScanner` ed espone `getFSReport` come funzione standalone tramite `bind`, nascondendo completamente l'esistenza della classe:

```
1 const scanner = FsStatScanner();
2 const getFSReport = scanner.getReport.bind(scanner);
```

Il consumer può importare `getFSReport` direttamente come funzione, senza mai istanziare né conoscere `FsStatScanner`. Questa scelta rispecchia lo stile funzionale prevalente nell'ecosistema Node.js, dove si preferisce esporre funzioni piuttosto che oggetti con stato esplicito.

CLI e misurazione delle prestazioni. Il modulo `main.ts` implementa la CLI: legge il path da `process.argv[2]` con fallback alla directory corrente (`process.cwd()`), configura i parametri (`maxSize = 100 MB`, `nb = 5`) tramite l'utilità `FileSize.megabyte()`, e misura il tempo di esecuzione con `benchmark()`. Quest'ultima funzione, basata su `process.hrtime.bigint()` per precisione al nanosecondo, restituisce sia il risultato che la durata in millisecondi, che vengono poi stampati in formato JSON indentato tramite `JSON.stringify(report, null, 2)`.

3.3 Design dell'implementazione RX

L'implementazione reattiva è articolata, in analogia con la versione a Virtual Thread, in due classi principali del package `lib`: `Report`, che modella il risultato della computazione, e `FSStatLib`, che costruisce ed esegue la pipeline reattiva. La classe `Main` funge da consumer minimale, limitandosi a sottoscrivere al risultato prodotto dalla libreria.

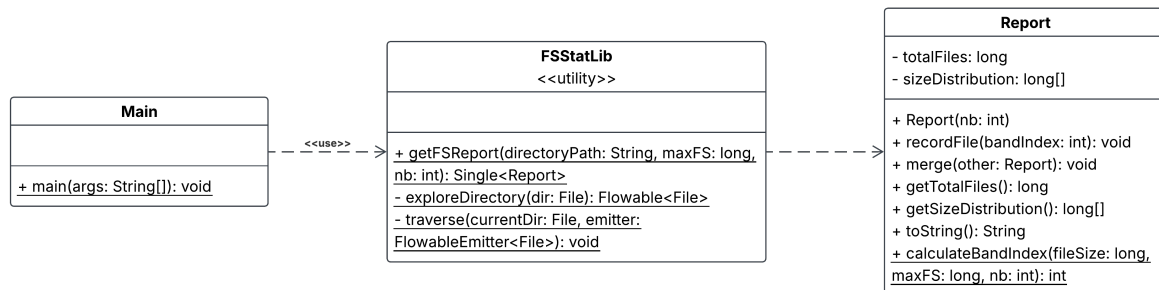


Figura 3: Diagramma UML delle classi dell'implementazione con java RX.

Il modello dati: Report. `Report` è un oggetto mutabile che incapsula il conteggio totale dei file (`totalFiles`) e un array `sizeDistribution` di dimensione `nb + 1`, dove le prime `nb` celle corrispondono alle bande regolari nell'intervallo `[0, MaxFS]` e l'ultima cella raccoglie i file che superano `MaxFS`. Il metodo statico `calculateBandIndex(fileSize, maxFS, nb)` calcola l'indice di banda tramite divisione intera sull'ampiezza di banda `maxFS / nb`, gestendo separatamente i due casi limite: dimensione superiore a `MaxFS` (indice `nb`) e dimensione esattamente pari a `MaxFS` (ultima banda regolare, indice `nb - 1`), per evitare un accesso fuori dai limiti dell'array che si verificherebbe con un semplice arrotondamento.

Come nelle altre due implementazioni, il metodo `merge(Report other)` è l'operazione cardine della strategia Map-Reduce: somma componente per componente le due distribuzioni e accumula i totali. Trattandosi di una somma, l'operazione è associativa e commutativa, il che la rende sicura da applicare in qualsiasi ordine durante le fasi di riduzione della pipeline.

Generazione dello stream: exploreDirectory. Il punto di partenza della pipeline è il metodo privato `exploreDirectory(File dir)`, che costruisce un `Flowable<File>` tramite `Flowable.create()`:

```

1 private static Flowable<File> exploreDirectory(File dir) {
2     return Flowable.create(emitter -> {
3         traverse(dir, emitter);
4         emitter.onComplete();
5     }, BackpressureStrategy.BUFFER);
6 }

```

La visita ricorsiva vera e propria è delegata al metodo di supporto `traverse()`, che scandisce l'albero di directory tramite `File.listFiles()`: per ogni sottodirectory richiama se stesso ricorsivamente, mentre per ogni file regolare invoca `emitter.onNext(file)`, immettendo l'elemento nello stream. A ogni iterazione viene verificato `emitter.isCancelled()`: se il consumer si è disiscritto (ad esempio tramite `dispose()` su una GUI che richiede l'interruzione), la visita si interrompe non appena possibile, evitando di proseguire un lavoro di cui nessuno è più interessato al risultato.

La strategia `BackpressureStrategy.BUFFER` è stata scelta perché la fase di scoperta dei file (l'unica parte della pipeline eseguita in modo strettamente sequenziale, dato che `traverse()`

non è parallelizzato) può produrre elementi più velocemente di quanto gli stadi a valle riescano a consumarli su alberi di directory molto popolati; il buffer disaccoppia produttore e consumatore accodando gli eventi in eccesso, a costo di un potenziale utilizzo di memoria proporzionale al numero di file non ancora elaborati.

Parallelizzazione della classificazione: `ParallelFlowable`. Una volta ottenuto lo stream sequenziale di `File`, l'operatore `parallel()` lo converte in un `ParallelFlowable`, suddividendo virtualmente il flusso in un numero di *rail* pari, di default, al numero di core disponibili. L'operatore `runOn(Schedulers.io())` assegna concretamente ciascun rail a un thread dello scheduler I/O di RxJava (un pool elastico, pensato per un elevato numero di operazioni brevi e potenzialmente bloccanti), che è dove viene effettivamente eseguito il lavoro di classificazione:

```
1 return exploreDirectory(rootDir)
2     .parallel()
3     .runOn(Schedulers.io())
4     .map(file -> {
5         Report partialReport = new Report(nb);
6         int bandIndex = Report.calculateBandIndex(file.length(), maxFS, nb);
7         partialReport.recordFile(bandIndex);
8         return partialReport;
9     })
10    .collect(() -> new Report(nb),
11            (railReport, partialReport) -> railReport.merge(partialReport))
12    .sequential()
13    .collect(() -> new Report(nb),
14            (mainReport, railReport) -> mainReport.merge(railReport));
```

Per ciascun file, l'operatore `map` produce un `Report` "puntuale", contenente il conteggio di un solo file già classificato nella banda corretta tramite `Report.calculateBandIndex()` e `recordFile()`. Questa scelta rende l'operazione di mapping pura e priva di effetti collaterali: ogni thread lavora esclusivamente su oggetti a sé locali, senza mai leggere o scrivere uno stato condiviso con gli altri rail.

Riduzione a due livelli. L'aggregazione dei report avviene in due fasi distinte, rispecchiando la struttura a "rail" del `ParallelFlowable`:

1. Il primo `collect()`, eseguito ancora in contesto parallelo, fonde i `Report` puntuali prodotti da ciascun rail in un unico `Report di rail`, tramite un accumulatore locale `() -> new Report(nb)` e la funzione di fusione `railReport.merge(partialReport)`. Poiché ogni rail possiede un proprio accumulatore privato, questa riduzione non richiede alcuna sincronizzazione: nessun altro thread accede contemporaneamente allo stesso `Report`.
2. L'operatore `sequential()` ricongiunge i rail paralleli in un unico `Flowable` standard, emettendo in sequenza i (al più n core) report parziali di rail.
3. Il secondo `collect()` esegue la riduzione finale, fondendo sequenzialmente tutti i report di rail in un unico `Report` conclusivo, restituito come `Single<Report>`.

Questa struttura a doppia riduzione è concettualmente identica al pattern Map-Reduce adottato dalle altre due versioni (fusione bottom-up di risultati locali), ma qui la "gerarchia" non rispecchia l'albero delle directory bensì la suddivisione in rail del `ParallelFlowable`: è la libreria RxJava, e non il programmatore, a decidere in quale rail e su quale thread ciascun elemento dello stream venga elaborato.

Assenza di sincronizzazione esplicita. Come nelle altre due implementazioni, in nessun punto della pipeline compaiono `synchronized`, `Lock` o variabili atomiche. La sicurezza rispetto alle race condition non deriva qui dall'assenza di stato condiviso (come nei Virtual Thread) né dalla *run-to-completion semantics* di un singolo thread (come nell'Event Loop), bensì da una garanzia strutturale di RxJava: gli operatori `collect()` invocano il proprio accumulatore in modo strettamente sequenziale all'interno di ciascun rail, e ogni rail possiede un accumulatore proprio, non condiviso con gli altri. La libreria si fa quindi carico di serializzare gli accessi allo stato mutabile del report, sollevando lo sviluppatore da qualsiasi responsabilità di sincronizzazione manuale.

Integrazione con la CLI: Main e ponte verso il modello bloccante. Il metodo pubblico `getFSReport()` restituisce un `Single<Report>`, il tipo RX più adatto a rappresentare una computazione asincrona che produce esattamente un risultato (o un errore), analogo concettualmente alla `CompletableFuture` della versione a Virtual Thread. In **Main**, la sottoscrizione avviene tramite `subscribe()` con due callback distinte, per il caso di successo e per il caso di errore:

```
1 reportSingle.subscribe(  
2     report -> { System.out.println(report); latch.countDown(); },  
3     error  -> { System.err.println(error.getMessage()); latch.countDown(); }  
4 );
```

Poiché la sottoscrizione è per sua natura non bloccante, il thread principale terminerebbe immediatamente senza attendere il risultato: per questo motivo **Main** introduce un `CountDownLatch(1)`, decrementato da entrambe le callback, e ne attende il rilascio con `latch.await()`. Questo meccanismo funge da ponte esplicito tra il mondo asincrono della pipeline reattiva e il modello di esecuzione sincrono di una CLI testuale, con un ruolo analogo a quello svolto da `.get()/.join()` nella versione a Virtual Thread.

Gestione degli errori. Se il percorso fornito non esiste o non è una directory, `getFSReport()` restituisce immediatamente un `Single.error()`, senza avviare alcuna elaborazione. Qualsiasi eccezione sollevata all'interno della pipeline (ad esempio durante l'accesso al file system) viene automaticamente incanalata da RxJava verso il canale di errore dello stream, raggiungendo la callback `onError` passata a `subscribe()` senza necessità di propagazione manuale tramite blocchi `try-catch` espliciti nel codice applicativo.